November 8, 2026 at 18:08

1. Introduction. An *autarky* for a family of clauses F is a set A of strictly distinct literals such that every clause of F either contains a literal of A or contains no literal of $\bar{A}$; in the words of page 228, "A satisfies every clause that A or $\bar{A}$ touches." Whenever A is an autarky we may set its literals true without losing satisfiability, which is why Algorithm X looks for autarkies while it looks ahead. Exercise 7.2.2.2–165 asks for the positive ones:

> **165. [26]** Design an algorithm to find the largest positive autarky A for a given F, namely an autarky that contains only positive literals. *Hint:* Warm up by finding the largest positive autarky for the clauses $\{12\bar{3}, 12\bar{5}, \bar{1}\bar{3}\bar{4}, 13\bar{6}, 1\bar{4}5, 156, \bar{2}35, 2\bar{4}6, 345, \bar{3}56\}$.

2. The answer is a small gem, and every sentence of it makes a claim.

1. In the warm-up, $\bar{1}\bar{3}\bar{4}$ excludes 1, 3, 4; then $13\bar{6}$ excludes 6; and $A = \{2, 5\}$ works, so it is the maximum.
2. A maximum (not just maximal) positive autarky always exists, because the union of positive autarkies is a positive autarky.
3. A clause $(v_1 \vee \cdots \vee v_s \vee \bar{v}_{s+1} \vee \cdots \vee \bar{v}_{s+t})$ of F says that $v_1 \notin A$ and $\cdots$ and $v_s \notin A$ imply $v_{s+j} \notin A$, so it generates t Horn clauses; their core is the set of positive literals that lie in no positive autarky.
4. A variant of Algorithm 7.1.1C, with steps C1 and C5 modified so that one clause of F yields t Horn clauses, finds that core in linear time.
5. By complementing some variables and prohibiting others, the same method finds the largest autarky contained in any given set of strictly distinct literals.

This program checks all five. The first four are statements about clauses and sets of variables, so they are checked against the definition by brute force wherever the number of variables allows it. For larger problems the `sat` package of the directory above supplies an independent route: a positive autarky containing v exists exactly when a certain set of clauses is satisfiable under the assumption a_v, and the package is asked that question once per variable. It is also asked, twice per problem, whether the autarky principle of page 228 really holds.

I wrote this on 16 September 2026.

3. Every claim is a mode.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/bits"
    "math/rand/v2"
    "os"
    "strconv"
    "strings"
    "github.com/sjnam/sat"
)
⟨ Types 8 ⟩
⟨ Global variables 5 ⟩
⟨ Functions 6 ⟩
func main() {
    ⟨ Read the command line 4 ⟩
    if on("warmup") {
        ⟨ Check the warm-up example 18 ⟩
    }
    if on("union") {
        ⟨ Check that positive autarkies are closed under union 21 ⟩
    }
    if on("core") {
        ⟨ Check the core algorithm against brute force 23 ⟩
    }
    if on("linear") {
        ⟨ Measure the running time of the core algorithm 25 ⟩
    }
    if on("within") {
        ⟨ Check the bracketed remark 29 ⟩
    }
    if on("solver") {
        ⟨ Check large problems with the sat package 31 ⟩
    }
    ⟨ Say how it went 7 ⟩
}
```

4. ⟨ Read the command line 4 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all",
    "warmup, union, core, linear, within, solver, or all")
flag.IntVar(&reps, "reps", 2000, "how many random cases per size")
flag.Uint64Var(&seed, "seed", 20260916, "seed for the random cases")
flag.Parse()
```

This code is used in section 3.

5. ⟨ Global variables 5 ⟩ ≡

```
var (
    mode string
    reps int
    seed uint64
    failures int
)
```

This code is used in section 3.

6. Each claim is announced, checked, and marked; one `FAIL` anywhere makes the program exit with a nonzero status.

⟨ Functions 6 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
func claim(ok bool, format string, args ... any) {
    mark := "ok␣␣"
    if ¬ok {
        mark = "FAIL"
        failures++
    }
    fmt.Printf("␣␣%s␣␣%s\n", mark, fmt.Sprintf(format, args...))
}
```

This code is also defined in sections 9, 10, 11, 12, 13, 17, 26, 27, 28, 32, 39.

This code is used in section 3.

7. ⟨ Say how it went 7 ⟩ ≡

```
if failures > 0 {
    fmt.Printf("\n%d␣claims␣failed.\n", failures)
    os.Exit(1)
}
fmt.Println("\nEvery␣claim␣checked␣out.")
```

This code is used in section 3.

8. Clauses and autarkies. Literals are numbered as Knuth numbers them, $2v$ for v and $2v+1$ for $\bar{v}$. The clauses in this program can be long and the problems can have a few hundred variables, so a clause is simply a list of literals.

⟨ Types 8 ⟩ ≡

```
type fam struct {
    n int
    cl [][]int
}
```

This code is used in section 3.

9. Knuth's shorthand '$12\bar{3}$' is read as `123'`, one digit per variable.

⟨ Functions 6 ⟩ +≡

```
func parseFam(n int, s string) fam {
    f := fam{n: n}
    for _, w := range strings.Fields(s) {
        var c []int
        for i := 0; i < len(w); i++ {
            l := 2 * int(w[i] - '0')
            if i + 1 < len(w) ∧ w[i + 1] ≡ '\'' {
                l, i = l + 1, i + 1
            }
            c = append(c, l)
        }
        f.cl = append(f.cl, c)
    }
    return f
}
func clauseName(c []int) string {
    var b strings.Builder
    for _, l := range c {
        b.WriteString(strconv.Itoa(l ≫ 1))
        if l & 1 ≡ 1 {
            b.WriteByte('\'')
        }
    }
    return b.String()
}
```

10. The definition, word for word. $in[l]$ says whether literal l belongs to A; a clause touched by A or $\bar{A}$ must contain a literal of A.

⟨ Functions 6 ⟩ +≡

```
func isAutarky(f fam, in []bool) bool {
    for _, c := range f.cl {
        touched, satisfied := false, false
        for _, l := range c {
            satisfied = satisfied ∨ in[l]
            touched = touched ∨ in[l] ∨ in[l ⊕ 1]
        }
        if touched ∧ ¬satisfied {
            return false
        }
    }
    return true
}
```

11. A positive autarky is a set of variables; for brute force it is a bitmask.

⟨ Functions 6 ⟩ +≡

```
func positive(n int, set uint64) []bool {
    in := make([]bool, 2 * n + 2)
    for v := 1; v ≤ n; v++ {
        if set & (1 ≪ v) ≠ 0 {
            in[2 * v] = true
        }
    }
    return in
}
```

12. Brute force runs through all 2^n sets of variables. It reports a largest positive autarky, the union of all of them, and whether that union is itself an autarky.

⟨ Functions 6 ⟩ +≡

```
func bruteMax(f fam)(best, union uint64, count int, unionOK bool) {
    for set := uint64(0); set < 1 ≪ uint(f.n); set++ {
        s := set ≪ 1
        if ¬isAutarky(f, positive(f.n, s)) {
            continue
        }
        count++
        union |= s
        if bits.OnesCount64(s) > bits.OnesCount64(best) {
            best = s
        }
    }
    return best, union, count, isAutarky(f, positive(f.n, union))
}
```

13. The core algorithm. Let the proposition P_v mean "$v \notin A$." A clause of F with positive variables $v_1, \ldots, v_s$ and negative variables $v_{s+1}, \ldots, v_{s+t}$ is satisfied by every positive autarky that touches it, so $P_{v_1} \wedge \cdots \wedge P_{v_s} \Rightarrow P_{v_{s+j}}$ for $1 \le j \le t$. These are definite Horn clauses, and Algorithm 7.1.1C finds their core.

Here is that algorithm with Knuth's own data structures. *conclusion*$[c]$, which in the original is a single proposition, becomes a list—that is the change to step C1—and step C5 then deduces every proposition on the list. Variables that are *prohibited* enter the core at the start; the answer's last remark needs that. *reason*$[v]$ records the clause that put v into the core, or -1 for a prohibited variable. *steps* counts one for each literal looked at in C1, each hypothesis in C4, and each conclusion in C5.

```
⟨ Functions 6 ⟩ +≡
  func core(f fam, prohibited []bool)(truth []bool, reason []int, steps int) {
      ⟨ Set up the data structures of Algorithm C 14 ⟩
      ⟨ Do step C1, with several conclusions per clause 15 ⟩
      for len(stack) > 0 {
          ⟨ Do steps C2 through C6 for the proposition on top of the stack 16 ⟩
      }
      return truth, reason, steps
  }
```

14. ⟨ Set up the data structures of Algorithm C 14 ⟩ ≡

```
truth, reason = make([]bool, f.n + 1), make([]int, f.n + 1)
last := make([]int, f.n + 1)
for v := range last {
    last[v], reason[v] = −1, −2
}
conclusion, count := make([][]int, len(f.cl)), make([]int, len(f.cl))
var hclause, hprev, stack []int
for v := 1; v ≤ f.n; v++ {
    if prohibited ≠ Λ ∧ prohibited[v] {
        truth[v], reason[v], stack = true, −1, append(stack, v)
    }
}
```

This code is used in section 13.

15. ⟨ Do step C1, with several conclusions per clause 15 ⟩ ≡

```
for c, cl := range f.cl {
    for _, l := range cl {
        steps++
        if l & 1 ≡ 1 {
            conclusion[c] = append(conclusion[c], l ≫ 1)
        } else {
            count[c]++
        }
    }
    if count[c] ≡ 0 {
        for _, v := range conclusion[c] {
            if ¬truth[v] {
                truth[v], reason[v], stack = true, c, append(stack, v)
            }
        }
        continue
    }
    for _, l := range cl {
        if l & 1 ≡ 0 {
            hclause, hprev = append(hclause, c), append(hprev, last[l ≫ 1])
            last[l ≫ 1] = len(hclause) − 1
        }
    }
}
```

This code is used in section 13.

16. ⟨ Do steps C2 through C6 for the proposition on top of the stack 16 ⟩ ≡

```
p := stack[len(stack) − 1]
stack = stack[:len(stack) − 1]
for h := last[p]; h ≥ 0; h = hprev[h] {
    steps++
    c := hclause[h]
    if count[c]−−; count[c] ≠ 0 {
        continue
    }
    for _, v := range conclusion[c] {
        steps++
        if ¬truth[v] {
            truth[v], reason[v], stack = true, c, append(stack, v)
        }
    }
}
```

This code is used in section 13.

17. The largest positive autarky is what the core leaves out.

⟨ Functions 6 ⟩ +≡

```
func largest(f fam) uint64 {
    truth, _, _ := core(f, Λ)
    var a uint64
    for v := 1; v ≤ f.n ∧ v < 64; v++ {
        if ¬truth[v] {
            a |= 1 ≪ v
        }
    }
    return a
}
```

18. The warm-up.

⟨ Check the warm-up example 18 ⟩ ≡

```
fmt.Println("165. the warm-up")
w := parseFam(6, "123' 125' 1'3'4' 136' 14'5 156 2'35 24'6 345 3'56")
best, union, count, unionOK := bruteMax(w)
claim(best ≡ 1 ≪ 2 | 1 ≪ 5 ∧ union ≡ best ∧ unionOK,
    "by brute force: %d positive autarkies, and the largest is {2,5}", count)
truth, reason, _ := core(w, Λ)
⟨ Replay the answer's deductions 19 ⟩
claim(largest(w) ≡ best, "the core algorithm finds {2,5} too")
⟨ Check the example in answer 157 20 ⟩
```

This code is used in section 3.

19. The answer's reasons are specific: 1, 3, and 4 because of $\bar{1}\bar{3}\bar{4}$, and 6 because of $13\bar{6}$. The core algorithm records a reason for every variable it excludes, so the program asks whether they are those.

⟨ Replay the answer's deductions 19 ⟩ ≡

```
var why []string
for v := 1; v ≤ 6; v++ {
    if truth[v] {
        why = append(why, fmt.Sprintf("%d by %s", v, clauseName(w.cl[reason[v]])))
    }
}
claim(strings.Join(why, ", ") ≡ "1 by 1'3'4', 3 by 1'3'4', 4 by 1'3'4', 6 by 136'",
    "excluded: %s", strings.Join(why, ", "))
```

This code is used in section 18.

20. Answer 157 gives $\{1, 2\}$ as an autarky of $\{1\bar{2}3, \bar{1}24, \bar{3}\bar{4}\}$ that is not a pure literal. It is positive, and it is the largest: $\bar{3}\bar{4}$ has no positive literal, so 3 and 4 are out.

⟨ Check the example in answer 157 20 ⟩ ≡

```
e157 := parseFam(4, "12'3 1'24 3'4'")
b157, _, _, _ := bruteMax(e157)
claim(b157 ≡ 1 ≪ 1 | 1 ≪ 2 ∧ largest(e157) ≡ b157,
    "answer 157's autarky {1,2} is the largest positive one of its clauses")
```

This code is used in section 18.

21. Unions. The answer's argument for a maximum is that the union of positive autarkies is a positive autarky. Checked on every family of at most five clauses on three variables, and on random families of up to ten variables.

The word *positive* matters. For general autarkies the union may not even be a set of strictly distinct literals: $F = \{12, \bar{1}\bar{2}\}$ has the two autarkies $\{1, \bar{2}\}$ and $\{\bar{1}, 2\}$, each of them maximal, and no maximum.

⟨ Check that positive autarkies are closed under union 21 ⟩ ≡

```
fmt.Println("165. unions of positive autarkies")
var nu, badU int
test := func(f fam) {
    nu++
    if best, union, _, ok := bruteMax(f); ¬ok ∨ best ≠ union {
        badU++
    }
}
everyFamily(3, 5, test)
sweep(10, reps/10, test)
claim(badU ≡ 0, "%d families, %d where the union fails", nu, badU)
⟨ Show that general autarkies have no maximum 22 ⟩
```

This code is used in section 3.

22. Brute force over all 3^n sets of strictly distinct literals.

⟨ Show that general autarkies have no maximum 22 ⟩ ≡

```
g := parseFam(2, "12 1'2'")
var maximal []string
for a := 0; a < 9; a++ {
    in := make([]bool, 6)
    for v := 1; v ≤ 2; v++ {
        if d := a/[]int { 1, 3 }[v − 1] % 3; d > 0{
            in[2 ∗ v + d − 1] = true
        }
    }
    if isAutarky(g, in) ∧ (in[2] ∨ in[3]) ∧ (in[4] ∨ in[5]) {
        maximal = append(maximal, fmt.Sprint(in[2:]))
    }
}
claim(len(maximal) ≡ 2, "{12, 1'2'} has two maximal autarkies and no maximum")
```

This code is used in section 21.

23. The core against brute force.

⟨ Check the core algorithm against brute force 23 ⟩ ≡

```
fmt.Println("165. the core algorithm")
var nc, badC, badSteps int
cmp := func(f fam) {
    nc++
    best, _, _, _ := bruteMax(f)
    if largest(f) ≠ best {
        badC++
    }
    ⟨ Check that the work is at most twice the total length 24 ⟩
}
everyFamily(3, 5, cmp)
sweep(12, reps/10, cmp)
claim(badC ≡ 0, "%d families: the core misses the largest positive autarky " +
    "%d times", nc, badC)
claim(badSteps ≡ 0, "steps never exceed twice the number of literals: " +
    "%d exceptions", badSteps)
```

This code is used in section 3.

24. Every literal is looked at once in C1. A proposition is pushed at most once, so each hypothesis is looked at most once in C4, and a clause's count reaches zero at most once, so each conclusion is looked at most once in C5. Hence the steps are at most $2N$, where N is the total length of the clauses.

⟨ Check that the work is at most twice the total length 24 ⟩ ≡

```
_, _, steps := core(f, Λ)
total := 0
for _, c := range f.cl {
    total += len(c)
}
if steps > 2 * total {
    badSteps++
}
```

This code is used in section 23.

25. Linear time. The point of changing steps C1 and C5, rather than simply writing out the t Horn clauses, shows up on long clauses. Take $2k$ variables, the one clause $(k+1 \vee \cdots \vee 2k \vee \bar{1} \vee \cdots \vee \bar{k})$, and the k unit clauses $\overline{k+1}$, …, $\overline{2k}$. The clause of length $2k$ becomes k Horn clauses with k hypotheses each, so writing them out costs k^2; the modified algorithm touches each literal a bounded number of times.

⟨ Measure the running time of the core algorithm 25 ⟩ ≡

```
fmt.Println("165. linear time")
var lastVariant, lastExpanded, lastN int
for k := 10; k ≤ 160; k *= 2 {
    f := wide(k)
    _, _, s1 := core(f, Λ)
    _, _, s2 := core(expand(f), Λ)
    lastVariant, lastExpanded, lastN = s1, s2, 3 * k
    fmt.Printf("         k=%3d: N=%4d, %5d steps modified, %6d written out\n",
        k, 3 * k, s1, s2)
}
claim(lastVariant ≤ 2 * lastN ∧ lastExpanded > 50 * lastN,
    "the modified algorithm stays within 2N; writing the clauses out does not")
```

This code is used in section 3.

26. ⟨ Functions 6 ⟩ +≡

```
func wide(k int) fam {
    f := fam{n: 2 * k}
    var big []int
    for v := k + 1; v ≤ 2 * k; v++ {
        big = append(big, 2 * v)
        f.cl = append(f.cl, []int{2 * v + 1})
    }
    for v := 1; v ≤ k; v++ {
        big = append(big, 2 * v + 1)
    }
    f.cl = append(f.cl, big)
    return f
}
```

27. Writing out the Horn clauses: one clause per negative literal, keeping all the positive ones. Each has a single conclusion, so on these the modified algorithm is Algorithm 7.1.1C itself.

⟨ Functions 6 ⟩ +≡

```
func expand(f fam) fam {
    g := fam{n: f.n}
    for _, c := range f.cl {
        var pos []int
        for _, l := range c {
            if l & 1 ≡ 0 {
                pos = append(pos, l)
            }
        }
        for _, l := range c {
            if l & 1 ≡ 1 {
                g.cl = append(g.cl, append(append([]int{}, pos...), l))
            }
        }
        if len(pos) ≡ len(c) {
            g.cl = append(g.cl, pos)
        }
    }
    return g
}
```

28. Autarkies inside a given set. The answer's last remark: complement the variables that appear negatively in a given set S of strictly distinct literals, prohibit the variables that do not appear in S at all, and the core algorithm finds the largest autarky contained in S.

⟨ Functions 6 ⟩ +≡

```
func within(f fam, S []int) []int {
    flip, prohibited := make([]bool, f.n + 1), make([]bool, f.n + 1)
    for v := range prohibited {
        prohibited[v] = true
    }
    for _, l := range S {
        prohibited[l ≫ 1], flip[l ≫ 1] = false, l & 1 ≡ 1
    }
    g := fam{n: f.n}
    for _, c := range f.cl {
        d := make([]int, len(c))
        for i, l := range c {
            if d[i] = l; flip[l ≫ 1] {
                d[i] = l ⊕ 1
            }
        }
        g.cl = append(g.cl, d)
    }
    truth, _, _ := core(g, prohibited)
    var a []int
    for _, l := range S {
        if ¬truth[l ≫ 1] {
            a = append(a, l)
        }
    }
    return a
}
```

29. Brute force tries every subset of S.

⟨ Check the bracketed remark 29 ⟩ ≡

```
fmt.Println("165. the largest autarky inside a given set of literals")
var nw, badW int
rw := rand.New(rand.NewPCG(seed, 165))
sweep(9, reps/5, func(f fam) {
    var S []int
    for v := 1; v ≤ f.n; v++ {
        if r := rw.IntN(3); r > 0 {
            S = append(S, 2 * v + r - 1)
        }
    }
    nw++
    ⟨ Compare within with the best subset of S 30 ⟩
})
claim(badW ≡ 0, "%d families with random sets S, %d disagreements", nw, badW)
```

This code is used in section 3.

30. ⟨ Compare *within* with the best subset of S 30 ⟩ ≡

```
bestSize, unionIn := −1, make([]bool, 2 ∗ f.n + 2)
for sub := 0; sub < 1 ≪ len(S); sub++ {
    in := make([]bool, 2 ∗ f.n + 2)
    for i, l := range S {
        in[l] = sub & (1 ≪ i) ≠ 0
    }
    if isAutarky(f, in) {
        bestSize = max(bestSize, bits.OnesCount(uint(sub)))
        for l := range in {
            unionIn[l] = unionIn[l] ∨ in[l]
        }
    }
}
got := within(f, S)
gotIn := make([]bool, 2 ∗ f.n + 2)
for _, l := range got {
    gotIn[l] = true
}
if len(got) ≠ bestSize ∨ ¬isAutarky(f, gotIn) ∨ fmt.Sprint(gotIn) ≠ fmt.Sprint(unionIn) {
    badW++
}
```

This code is used in section 29.

31. The sat package. Brute force stops at a dozen variables. For larger problems the program plants a positive autarky H in a random 3SAT problem—it draws clauses at random and throws away those that H touches without satisfying—and then asks two questions of the solver.

First, is the result of the core algorithm really the largest? Introduce a variable a_v for each v, meaning $v \in A$, and for each clause of F and each negative literal $\bar{w}$ in it, the clause $(\bar{a}_w \vee a_{v_1} \vee \cdots \vee a_{v_s})$. The satisfying assignments of these clauses are exactly the positive autarkies, so a positive autarky containing v exists if and only if they are satisfiable under the assumption a_v. That is a different computation from the core—a search rather than a propagation—though the clauses are dual Horn and the solver never has to work hard. Each model it returns is checked against the definition as well.

Second, the autarky principle. If A is an autarky, F is satisfiable if and only if the clauses that A does not touch are satisfiable; and a model of those, with the literals of A made true, satisfies F. The solver is asked both questions and the model is checked.

⟨ Check large problems with the `sat` package 31 ⟩ ≡

```
fmt.Println("larger␣problems,␣with␣the␣sat␣package")
var np, calls, badMax, badModel, badPlant, badPrinciple, badExtend, sats, unsats int
var badDef, sizeH, sizeA int
rp := rand.New(rand.NewPCG(seed, 228))
for i := 0; i < reps/40; i++ {
    n := 60 + rp.IntN(80)
    f, H := planted(rp, n)
    np++
    truth, _, _ := core(f, Λ)
    ⟨ Check the answer against the definition, and measure it 33 ⟩
    ⟨ Check that H lies inside the largest positive autarky 34 ⟩
    ⟨ Ask the solver whether each variable can be in a positive autarky 35 ⟩
    ⟨ Test the autarky principle with the solver 37 ⟩
}
claim(badDef ≡ 0, "%d␣problems␣with␣60␣to␣139␣variables:␣every␣result␣is␣" +
    "a␣positive␣autarky␣by␣the␣definition", np)
claim(badPlant ≡ 0, "the␣planted␣autarkies␣(%d␣variables␣in␣all)␣always␣lie␣" +
    "inside␣the␣ones␣found␣(%d)", sizeH, sizeA)
claim(badMax ≡ 0 ∧ badModel ≡ 0, "%d␣calls␣with␣assumptions␣agree␣with␣the␣" +
    "core,␣and␣every␣model␣is␣a␣positive␣autarky", calls)
claim(badPrinciple ≡ 0 ∧ badExtend ≡ 0 ∧ sats > 0 ∧ unsats > 0,
    "the␣autarky␣principle␣holds:␣%d␣satisfiable,␣%d␣unsatisfiable,␣" +
      "every␣extended␣model␣satisfies␣F", sats, unsats)
```

This code is used in section 3.

32. ⟨ Functions 6 ⟩ +≡

```
func planted(rng *rand.Rand, n int)(fam, []bool) {
    H := make([]bool, n + 1)
    for v := 1; v ≤ n; v++ {
        H[v] = rng.IntN(10) < 3
    }
    f := fam{n: n}
    for len(f.cl) < 6 * n {
        c := randClause(rng, n, 3)
        touched, satisfied := false, false
        for _, l := range c {
            touched = touched ∨ H[l ≫ 1]
            satisfied = satisfied ∨ (H[l ≫ 1] ∧ l & 1 ≡ 0)
        }
        if ¬touched ∨ satisfied {
            f.cl = append(f.cl, c)
        }
    }
    return f, H
}
```

33. ⟨ Check the answer against the definition, and measure it 33 ⟩ ≡

```
inA := make([]bool, 2 * n + 2)
for v := 1; v ≤ n; v++ {
    if inA[2 * v] = ¬truth[v]; ¬truth[v] {
        sizeA++
    }
    if H[v] {
        sizeH++
    }
}
if ¬isAutarky(f, inA) {
    badDef++
}
```

This code is used in section 31.

34. ⟨ Check that H lies inside the largest positive autarky 34 ⟩ ≡

```
for v := 1; v ≤ n; v++ {
    if H[v] ∧ truth[v] {
        badPlant++
        break
    }
}
```

This code is used in section 31.

35. ⟨ Ask the solver whether each variable can be in a positive autarky 35 ⟩ ≡

```
s := sat.New()
for v := 1; v ≤ n; v++ {
    s.Lookup(strconv.Itoa(v))
}
for _, c := range f.cl {
    var pos []sat.Lit
    for _, l := range c {
        if l & 1 ≡ 0 {
            pos = append(pos, sat.Lit(l))
        }
    }
    for _, l := range c {
        if l & 1 ≡ 1 {
            s.AddClause(append([]sat.Lit{sat.Lit(l)}, pos...)...)
        }
    }
}
for v := 1; v ≤ n; v++ {
    st, _ := s.Solve(context.Background(), sat.Lit(2 * v))
    calls++
    if (st ≡ sat.Sat) ≡ truth[v] {
        badMax++
    }
    if st ≡ sat.Sat {
        ⟨ Check that the model is a positive autarky 36 ⟩
    }
}
```

This code is used in section 31.

36. In this solver the literal $2v$ stands for a_v, so $\bar{w} \vee \cdots$ is written with the literal $2w + 1$.

⟨ Check that the model is a positive autarky 36 ⟩ ≡

```
in := make([]bool, 2 * n + 2)
for u := 1; u ≤ n; u++ {
    in[2 * u] = s.Value(sat.Lit(2 * u))
}
if ¬isAutarky(f, in) {
    badModel++
}
```

This code is used in section 35.

37. ⟨ Test the autarky principle with the solver 37 ⟩ ≡

```
whole, rest := sat.New(), sat.New()
for v := 1; v ≤ n; v++ {
    whole.Lookup(strconv.Itoa(v))
    rest.Lookup(strconv.Itoa(v))
}
for _, c := range f.cl {
    ls, touched := make([]sat.Lit, len(c)), false
    for i, l := range c {
        ls[i], touched = sat.Lit(l), touched ∨ ¬truth[l ≫ 1]
    }
    whole.AddClause(ls...)
    if ¬touched {
        rest.AddClause(ls...)
    }
}
st1, _ := whole.Solve(context.Background())
st2, _ := rest.Solve(context.Background())
if st1 ≠ st2 {
    badPrinciple++
}
if st2 ≡ sat.Sat {
    sats++
    ⟨ Extend the model of the untouched clauses and check it 38 ⟩
} else {
    unsats++
}
```

This code is used in section 31.

38. ⟨ Extend the model of the untouched clauses and check it 38 ⟩ ≡

```
for _, c := range f.cl {
    ok := false
    for _, l := range c {
        v := l ≫ 1
        val := ¬truth[v] ∨ rest.Value(sat.Lit(2 * v))
        ok = ok ∨ (val ≡ (l & 1 ≡ 0))
    }
    if ¬ok {
        badExtend++
        break
    }
}
```

This code is used in section 37.

39. Random families. *sweep* draws random families on $1,\ldots,maxN$ variables; *everyFamily* produces every family of at most *max* clauses on n variables, from the 3^n possible clauses, the empty one included.

⟨ Functions 6 ⟩ +≡

```
func randClause(rng *rand.Rand, n, k int) []int {
    var c []int
    used := map[int]bool{}
    for len(c) < k ∧ len(c) < n {
        if v := 1 + rng.IntN(n); ¬used[v] {
            used[v], c = true, append(c, 2 * v + rng.IntN(2))
        }
    }
    return c
}
func sweep(maxN, count int, do func(fam)) {
    for n := 1; n ≤ maxN; n++ {
        rng := rand.New(rand.NewPCG(seed, uint64(n)))
        for i := 0; i < count; i++ {
            f := fam{n: n}
            for j, m := 0, 1 + rng.IntN(3 * n); j < m; j++ {
                f.cl = append(f.cl, randClause(rng, n, 1 + rng.IntN(4)))
            }
            do(f)
        }
    }
}
func everyFamily(n, max int, do func(fam)) {
    cands := [][][]int{Λ}
    for v := 1; v ≤ n; v++ {
        var next [][][]int
        for _, c := range cands {
            next = append(next, c, append(append([][]int{}, c...), 2 * v),
                append(append([][]int{}, c...), 2 * v + 1))
        }
        cands = next
    }
    var rec func(start int, cl [][][]int)
    rec = func(start int, cl [][][]int) {
        if len(cl) > 0 {
            do(fam{n: n, cl: cl})
        }
        if len(cl) ≡ max {
            return
        }
        for i := start; i < len(cands); i++ {
            rec(i + 1, append(append([][][]int{}, cl...), cands[i]))
        }
    }
    rec(0, Λ)
}
```

40. Index.

a: 17, 22, 28.
AddClause: 35, 37.
append: 9, 14, 15, 16, 19, 22, 26, 27, 28, 29, 32, 35, 39.
args: 6.
b: 9.
Background: 35, 37.
badC: 23.
badDef: 31, 33.
badExtend: 31, 38.
badMax: 31, 35.
badModel: 31, 36.
badPlant: 31, 34.
badPrinciple: 31, 37.
badSteps: 23, 24.
badU: 21.
badW: 29, 30.
best: 12, 18, 21, 23.
bestSize: 30.
big: 26.
bits: 12, 30.
bruteMax: 12, 18, 20, 21, 23.
Builder: 9.
b157: 20.
c: 9, 10, 13, 15, 16, 24, 27, 28, 32, 35, 37, 38, 39.
calls: 31, 35.
cands: 39.
cl: 8, 9, 10, 14, 15, 19, 24, 26, 27, 28, 32, 35, 37, 38, 39.
claim: 6, 18, 19, 20, 21, 22, 23, 25, 29, 31.
clauseName: 9, 19.
cmp: 23.
conclusion: 13, 14, 15, 16.
context: 35, 37.
core: 13, 17, 18, 24, 25, 28, 31.
count: 12, 14, 15, 16, 18, 39.
d: 22, 28.
do: 39.
everyFamily: 21, 23, 39.
Exit: 7.
expand: 25, 27.
e157: 20.
f: 9, 10, 12, 13, 14, 15, 17, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 35, 36, 37, 38, 39.
failures: 5, 6, 7.
fam: 8, 9, 10, 12, 13, 17, 21, 23, 26, 27, 28, 29, 32, 39.
Fields: 9.
flag: 4.
flip: 28.
fmt: 6, 7, 18, 19, 21, 22, 23, 25, 29, 30, 31.
format: 6.
g: 22, 27, 28.
got: 30.
gotIn: 30.
H: 31, 32, 33, 34.
h: 16.
hclause: 14, 15, 16.
hprev: 14, 15, 16.
i: 9, 28, 30, 31, 37, 39.
in: 10, 11, 22, 30, 36.
inA: 33.
IntN: 29, 31, 32, 39.
IntVar: 4.
isAutarky: 10, 12, 22, 30, 33, 36.
Itoa: 9, 35, 37.
j: 39.
Join: 19.
k: 25, 26, 39.
l: 9, 10, 15, 27, 28, 30, 32, 35, 37, 38.
largest: 17, 18, 20, 23.
last: 14, 15, 16.
lastExpanded: 25.
lastN: 25.
lastVariant: 25.
len: 9, 13, 14, 15, 16, 22, 24, 27, 28, 30, 32, 37, 39.
Lit: 35, 36, 37, 38.
Lookup: 35, 37.
ls: 37.
m: 6, 39.
main: 3.
make: 11, 14, 22, 28, 30, 32, 33, 36, 37.
mark: 6.
max: 30, 39.
maximal: 22.
maxN: 39.
mode: 4, 5, 6.
n: 8, 9, 11, 12, 14, 17, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 39.
nc: 23.
New: 29, 31, 35, 37, 39.
NewPCG: 29, 31, 39.
next: 39.
np: 31.

nu: 21.
nw: 29.
ok: 6, 21, 38.
on: 3, 6.
OnesCount: 30.
OnesCount64: 12.
os: 7.
p: 16.
Parse: 4.
parseFam: 9, 18, 20, 22.
planted: 31, 32.
pos: 27, 35.
positive: 11, 12.
Printf: 6, 7, 25.
Println: 7, 18, 21, 23, 25, 29, 31.
prohibited: 13, 14, 28.
r: 29.
Rand: 32, 39.
rand: 29, 31, 32, 39.
randClause: 32, 39.
reason: 13, 14, 15, 16, 18, 19.
rec: 39.
reps: 4, 5, 21, 23, 29, 31.
rest: 37, 38.
rng: 32, 39.
rp: 31.
rw: 29.
S: 28, 29, 30.
s: 9, 12, 35, 36.
Sat: 35, 37.
sat: 35, 36, 37, 38.
satisfied: 10, 32.
sats: 31, 37.
seed: 4, 5, 29, 31, 39.
set: 11, 12.
sizeA: 31, 33.
sizeH: 31, 33.
Solve: 35, 37.
Sprint: 22, 30.
Sprintf: 6, 19.
st: 35.
stack: 13, 14, 15, 16.
start: 39.
steps: 13, 15, 16, 24.
strconv: 9, 35, 37.
String: 9.
strings: 9, 19.
StringVar: 4.
st1: 37.
st2: 37.
sub: 30.
sweep: 21, 23, 29, 39.
s1: 25.
s2: 25.
test: 21.
total: 24.
touched: 10, 32, 37.
truth: 13, 14, 15, 16, 17, 18, 19, 28, 31, 33, 34, 35, 37, 38.
u: 36.
Uint64Var: 4.
union: 12, 18, 21.
unionIn: 30.
unionOK: 12, 18.
unsats: 31, 37.
used: 39.
v: 11, 13, 14, 15, 16, 17, 19, 22, 26, 28, 29, 32, 33, 34, 35, 37, 38, 39.
val: 38.
Value: 36, 38.
w: 9, 18, 19.
whole: 37.
why: 19.
wide: 25, 26.
within: 28, 30.
WriteByte: 9.
WriteString: 9.

⟨ Ask the solver whether each variable can be in a positive autarky 35 ⟩ Used in section 31
⟨ Check large problems with the `sat` package 31 ⟩ Used in section 3
⟨ Check that positive autarkies are closed under union 21 ⟩ Used in section 3
⟨ Check that the model is a positive autarky 36 ⟩ Used in section 35
⟨ Check that the work is at most twice the total length 24 ⟩ Used in section 23
⟨ Check that H lies inside the largest positive autarky 34 ⟩ Used in section 31
⟨ Check the answer against the definition, and measure it 33 ⟩ Used in section 31
⟨ Check the bracketed remark 29 ⟩ Used in section 3
⟨ Check the core algorithm against brute force 23 ⟩ Used in section 3
⟨ Check the example in answer 157 20 ⟩ Used in section 18
⟨ Check the warm-up example 18 ⟩ Used in section 3
⟨ Compare *within* with the best subset of S 30 ⟩ Used in section 29
⟨ Do step C1, with several conclusions per clause 15 ⟩ Used in section 13
⟨ Do steps C2 through C6 for the proposition on top of the stack 16 ⟩ Used in section 13
⟨ Extend the model of the untouched clauses and check it 38 ⟩ Used in section 37
⟨ Functions 6 ⟩ Used in section 3
⟨ Global variables 5 ⟩ Used in section 3
⟨ Measure the running time of the core algorithm 25 ⟩ Used in section 3
⟨ Read the command line 4 ⟩ Used in section 3
⟨ Replay the answer's deductions 19 ⟩ Used in section 18
⟨ Say how it went 7 ⟩ Used in section 3
⟨ Set up the data structures of Algorithm C 14 ⟩ Used in section 13
⟨ Show that general autarkies have no maximum 22 ⟩ Used in section 21
⟨ Test the autarky principle with the solver 37 ⟩ Used in section 31
⟨ Types 8 ⟩ Used in section 3

The largest positive autarky

	Section	Page
Introduction	1	1
Clauses and autarkies	8	4
The core algorithm	13	6
The warm-up	18	9
Unions	21	10
The core against brute force	23	11
Linear time	25	12
Autarkies inside a given set	28	14
The `sat` package	31	16
Random families	39	20
Index	40	21